



Guide pratique · v1.0

Manuel utilisateur

MOTO Editor

Guide complet pour découvrir et maîtriser MOTO Editor : installation, interface, éditeur, IA, paramètres et raccourcis clavier.

Septembre 2026 · Éditeur IA local-first

Sommaire

1. Installation et premiers pas	p.3
2. Interface et navigation	p.4
3. Éditeur de code	p.5
4. Terminal intégré	p.6
5. Panneaux IA	p.7
6. Paramètres	p.8
7. Raccourcis clavier	p.9
8. Dépannage et FAQ	p.10

Bienvenue dans MOTO Editor

MOTO Editor est un éditeur de code ultra-léger, entièrement local, conçu pour offrir une expérience moderne, rapide et assistée par IA — sans dépendances cloud, sans frameworks lourds, sans Electron.

Il combine :

- Une UI minimaliste et moderne
- Une IA locale via Ollama
- Une IA structurée via XENO-SSS[∞]
- Un terminal intégré
- Un système de paramètres data-driven (~300 options)

1. Installation et premiers pas

Prérequis

- **Système d'exploitation** : Windows 10/11 (version 1809 ou ultérieure)
- **Espace disque** : 200 Mo minimum
- **RAM** : 4 Go recommandé
- **IA locale (optionnel)** : Ollama installé pour activer les fonctionnalités IA

Installation

1. Téléchargez le fichier MotoEditor-Setup.exe depuis le site officiel.
2. Exécutez l'installateur et suivez les instructions.
3. Lancez MOTO Editor depuis le menu Démarrer ou le raccourci bureau.

💡 **Astuce** : Si vous souhaitez utiliser les fonctionnalités IA, installez Ollama (ollama.com) et téléchargez un modèle (ex: `ollama pull codellama`) avant de lancer MOTO Editor.

Premier lancement

Au premier lancement, MOTO Editor affiche l'écran d'accueil avec :

- Un explorateur de fichiers à gauche (dock gauche)
- La zone d'édition au centre
- Les panneaux IA à droite (dock droit)
- Le terminal en bas (dock du bas, replié par défaut)

Vous pouvez ouvrir un dossier de projet via **Fichier** → **Ouvrir un dossier** ou en glissant-déposant un dossier sur la fenêtre.

Importer un projet existant

MOTO Editor peut importer des projets depuis Visual Studio ou VS Code :

1. Cliquez sur **Fichier** → **Importer un projet**.
2. Sélectionnez le dossier contenant le projet (.sln, .csproj, ou dossier VS Code).
3. MOTO Editor détecte automatiquement la configuration et importe les paramètres.

2. Interface et navigation

Vue d'ensemble

L'interface de MOTO Editor est organisée en **3 docks redimensionnables** autour de la zone d'édition centrale :

- **Dock gauche** : Explorateur de fichiers, recherche dans le projet
- **Dock droit** : Panneaux IA (Cortex, Neural, Workspace, AutoLink, Context, etc.)
- **Dock du bas** : Terminal intégré, sortie de compilation, diagnostics

Redimensionner les docks

Glissez les bordures des docks pour les redimensionner. Les docks sont :

- **Redimensionnables** : glissez les bordures horizontales/verticales
- **Repliables** : double-cliquez sur la bordure pour replier/déplier
- **Glisser-déposer** : déplacez les panneaux d'un dock à l'autre

💡 **Astuce** : Pour masquer temporairement tous les docks et passer en mode "focus", appuyez sur **Ctrl+Shift+F**.

Barre de statut

La barre de statut en bas de la fenêtre affiche :

- Le mode de performance actif (Eco/Balanced/Turbo/Ultra)
- Le nombre d'erreurs/warnings dans le projet
- La position du curseur (ligne, colonne)
- Le langage du fichier actif
- L'état de la connexion IA (locale/distante)

Palette de commandes

La palette de commandes permet d'accéder rapidement à toutes les fonctionnalités :

- Raccourci : **Ctrl+Shift+P**
- Tapez pour filtrer les commandes
- Appuyez sur Entrée pour exécuter

3. Éditeur de code

Fonctionnalités principales

- **Onglets** : ouvrez plusieurs fichiers, badges d'erreurs, flèches ←/→ pour naviguer
- **Split** : divisez l'édition en 2 ou 4 volets (Ctrl+\)
- **Minimap** : vue d'ensemble du fichier à droite, interactive
- **Ghost text** : suggestions IA affichées en gris, appuyez sur Tab pour accepter
- **Coloration syntaxique** : supporte C#, JavaScript, Python, et 50+ langages
- **Breadcrumb** : navigation rapide dans la structure du fichier

Onglets et navigation

Gérez vos fichiers ouverts avec les onglets :

- **Fermer** : clic sur la croix ou Ctrl+W
- **Fermer tous sauf actif** : clic droit → Fermer les autres
- **Épingler** : clic droit → Épingler (l'onglet reste à gauche)
- **Navigation** : Ctrl+Tab pour basculer entre onglets

Split et multi-édition

Divisez l'écran pour travailler sur plusieurs fichiers simultanément :

- **Split vertical** : Ctrl+\ puis V
- **Split horizontal** : Ctrl+\ puis H
- **4 volets** : Ctrl+\ puis 4
- **Fermer le split** : Ctrl+\ puis X

Minimap interactive

La minimap à droite affiche une vue d'ensemble du fichier :

- Cliquez pour naviguer rapidement
- Glissez pour faire défiler
- Les erreurs et warnings sont marqués en rouge/orange
- Masquez-la via **Affichage** → **Minimap** ou les paramètres

4. Terminal intégré

Ouvrir le terminal

Le terminal est intégré dans le dock du bas :

- **Raccourci** : Ctrl+` (accent grave)
- **Menu** : Affichage → Terminal
- **Auto-ouverture** : le terminal s'ouvre automatiquement au lancement d'un projet

Utilisation

Le terminal supporte toutes les commandes shell standard :

```
# Exemples de commandes
dotnet build
dotnet run
npm install
git status
```

- **Historique** : flèches haut/bas pour naviguer dans l'historique
- **Coloration** : les erreurs s'affichent en rouge, les sorties normales en blanc
- **Effacer** : Ctrl+L ou commande clear

💡 **Astuce** : Le terminal est connecté à TerminalService. Les commandes sont historisées et persistées entre les sessions.

Sortie de compilation

Lorsque vous compilez un projet (Ctrl+Shift+B), la sortie s'affiche dans le terminal :

- Les erreurs sont colorées en rouge avec le fichier et la ligne concernés
- Les warnings sont colorés en orange
- Cliquez sur une erreur pour ouvrir le fichier à la ligne concernée

Diagnostics

Le panneau de diagnostics affiche les erreurs et warnings en temps réel :

- **Ouvrir** : Ctrl+Shift+M ou Affichage → Diagnostics
- Filtrez par type (erreurs, warnings, infos)
- Cliquez pour naviguer vers le fichier concerné

5. Panneaux IA

Vue d'ensemble

Les panneaux IA sont accessibles dans le dock droit. Chaque panneau offre une fonctionnalité spécifique :

- **Cortex** : mémoire cognitive, apprentissage de votre style
- **Neural** : recherche sémantique dans le projet
- **Workspace** : suggestions proactives contextuelles
- **AutoLink** : détection et résolution de références manquantes
- **Context** : suggestions senior proactives
- **Story Mode** : narration interactive du code
- **Health** : santé du projet, complexité cyclomatique
- **Time Machine** : snapshots granulaires sans Git
- **Evolution** : suivi de l'évolution architecturale

Cortex — Mémoire cognitive

Cortex apprend de vos habitudes et adapte les suggestions :

- Style de code préféré (indentation, nommage)
- Patterns récurrents
- Préférences de navigation

Les données sont persistées dans `.moto/cortex/memory.json`.

Neural — Recherche sémantique

Neural utilise des embeddings pour rechercher du code par signification :

- Tapez une description en langage naturel
- Neural trouve les fichiers et fonctions correspondants
- Utile pour retrouver du code oublié ou comprendre un projet inconnu

AutoLink — Références manquantes

AutoLink détecte les références manquantes et propose des corrections :

- Imports manquants
- Types non résolus
- Dépendances absentes

Cliquez sur **Apply** pour appliquer la correction, ou **Dismiss** pour ignorer.

6. Paramètres

Ouvrir les paramètres

Accédez aux paramètres via :

- **Raccourci** : Ctrl+I,
- **Menu** : Fichier → Préférences → Paramètres
- **Icône** : cliquez sur l'icône  dans la barre de statut

Recherche instantanée

Les paramètres sont organisés en 15 catégories, mais vous pouvez rechercher instantanément :

- Tapez dans la barre de recherche en haut
- Les paramètres sont filtrés en temps réel
- Exemples : "thème", "police", "minimap", "IA", "terminal"

Catégories principales

- **Général** : langue, mise à jour automatique, démarrage
- **Apparence** : thème, police, taille de police, minimap
- **Keymap** : raccourcis clavier personnalisables
- **Éditeur** : indentation, retour à la ligne, numéros de ligne
- **Langages & Outils** : paramètres spécifiques par langage
- **Recherche & Fichiers** : exclusions, filtrage
- **Fenêtre & Layout** : docks, panneaux, mode focus
- **Debugger** : points d'arrêt, variables, pile d'appels
- **Terminal** : shell, historique, coloration
- **Version Control** : Git, diff, merge
- **Collaboration** : sessions temps réel, présence
- **IA** : provider, suggestions, modes
- **Réseau** : proxy, timeout
- **Développeur** : logs, debug, expérimental

 **Astuce** : Les paramètres sont persistés dans %AppData%/MotoEditor/settings.json. Vous pouvez les sauvegarder et les restaurer facilement.

Templates de configuration

MOTO Editor propose des templates préconfigurés :

- **Débutant** : interface simplifiée, suggestions IA maximales
- **Intermédiaire** : équilibre entre simplicité et puissance
- **Expert** : interface minimaliste, contrôle total

Appliquez un template via **Paramètres** → **Templates**.

7. Raccourcis clavier

Navigation

Action	Raccourci
Palette de commandes	Ctrl+Shift+P
Ouvrir un fichier	Ctrl+P
Rechercher dans le fichier	Ctrl+F
Rechercher dans le projet	Ctrl+Shift+F
Remplacer	Ctrl+H
Aller à la ligne	Ctrl+G
Navigation arrière/avant	Alt+← / Alt+→

Édition

Action	Raccourci
Couper/Copier/Coller	Ctrl+X / Ctrl+C / Ctrl+V
Annuler/Rétablir	Ctrl+Z / Ctrl+Y
Dupliquer la ligne	Ctrl+D
Supprimer la ligne	Ctrl+Shift+K
Déplacer la ligne haut/bas	Alt+↑ / Alt+↓
Commenter/Décommenter	Ctrl+/
Indenter/Désindenter	Tab / Shift+Tab

Terminal et IA

Action	Raccourci
Ouvrir le terminal	Ctrl+`

Ouvrir les paramètres	Ctrl+,
Accepter la suggestion IA	Tab
Rejeter la suggestion IA	Esc
Ouvrir le panneau IA	Ctrl+Shift+I

Fichiers et projets

Action	Raccourci
Nouveau fichier	Ctrl+N
Ouvrir un dossier	Ctrl+Shift+O
Sauvegarder	Ctrl+S
Sauvegarder tout	Ctrl+Shift+S
Fermer l'onglet	Ctrl+W
Compiler	Ctrl+Shift+B
Lancer le projet	F5

8. Dépannage et FAQ

Problèmes courants

L'IA ne répond pas

- Vérifiez qu'Ollama est installé et en cours d'exécution
- Vérifiez qu'un modèle est téléchargé (`ollama list`)
- Vérifiez les paramètres IA dans **Paramètres** → **IA**

Le terminal ne s'ouvre pas

- Vérifiez que le dock du bas n'est pas replié (double-cliquez sur la bordure)
- Essayez le raccourci `Ctrl+``
- Vérifiez les paramètres du terminal dans **Paramètres** → **Terminal**

Les paramètres ne sont pas sauvegardés

- Vérifiez que le dossier `%AppData%/MotoEditor/` est accessible en écriture
- Vérifiez les permissions du fichier `settings.json`

FAQ

MOTO Editor fonctionne-t-il sans Internet ?

Oui, MOTO Editor est entièrement local. Les fonctionnalités IA utilisent Ollama en local. Seules les mises à jour et les fonctionnalités de collaboration nécessitent Internet.

Puis-je utiliser MOTO Editor sur macOS ou Linux ?

Actuellement, MOTO Editor est disponible uniquement sur Windows. Le support macOS et Linux est prévu pour la version 1.1.

Comment signaler un bug ?

Utilisez GitHub Issues : github.com/moto-editor/moto-editor/issues. Incluez :

- La version de MOTO Editor
- Votre système d'exploitation
- Les étapes pour reproduire le bug
- Les logs si disponibles (**Aide** → **Exporter les logs**)

Comment contribuer au projet ?

MOTO Editor est open-source. Consultez CONTRIBUTING.md sur GitHub pour les guidelines de contribution.

© 2026 MOTO Editor · Manuel utilisateur v1.0 · Septembre 2026
Généré le 2 septembre 2026